

**UNIVERSIDAD POLITÉCNICA DE
SAN LUIS POTOSÍ**

CARRERA: Ingeniería en Tecnologías de la Información

MATERIA: CNO IV Seguridad Informática

TRABAJO: FILE PATH TRAVERSAL SIMPLE CASE

Ojeda Camacho José Carlos / 181629

PROFESOR: Servando Contreras

Contenido

Información del laboratorio	2
Objetivo del laboratorio.....	2
Explicación técnica de la vulnerabilidad	2
Procedimiento paso a paso	3
Explicación del payload	7
Mitigación recomendada.....	7

Información del laboratorio

Nombre del laboratorio: **File path traversal, simple case**

Nivel (Apprentice / Practitioner): Apprentice

Tipo de explotación:

Objetivo del laboratorio

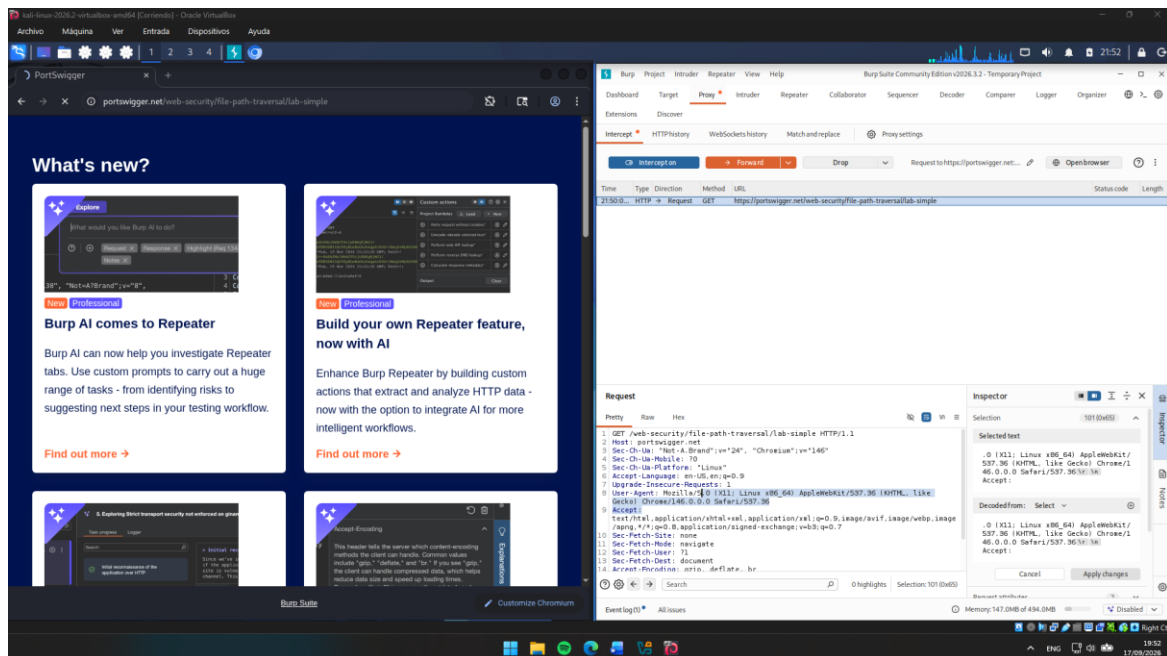
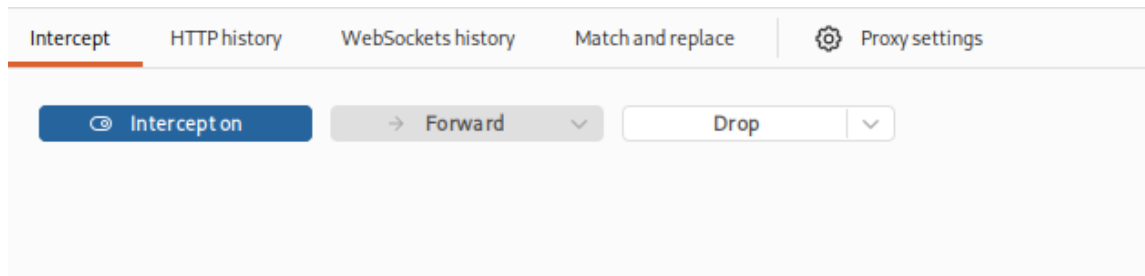
El objetivo de este laboratorio es explotar una vulnerabilidad de Path Traversal en la funcionalidad de visualización de imágenes de productos de la aplicación web, con el fin de leer y recuperar el contenido del archivo `/etc/passwd` del servidor.

Explicación técnica de la vulnerabilidad

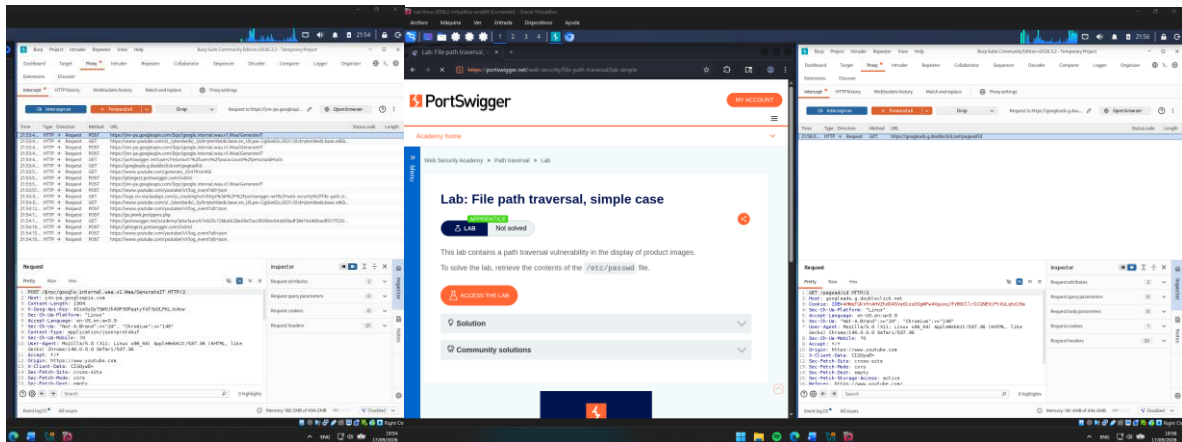
La vulnerabilidad de File Path Traversal (o Directory Traversal) ocurre cuando una aplicación web utiliza la entrada del usuario para construir la ruta a un archivo o directorio sin la validación o sanitización adecuada. Esto permite a un atacante manipular las referencias a los archivos (utilizando secuencias como `../`) para acceder a archivos y directorios que se encuentran fuera de la carpeta raíz de la web. En este escenario, el parámetro `filename` de la URL confía ciegamente en el input proporcionado, lo que permite leer archivos sensibles del sistema operativo del servidor.

Procedimiento paso a paso

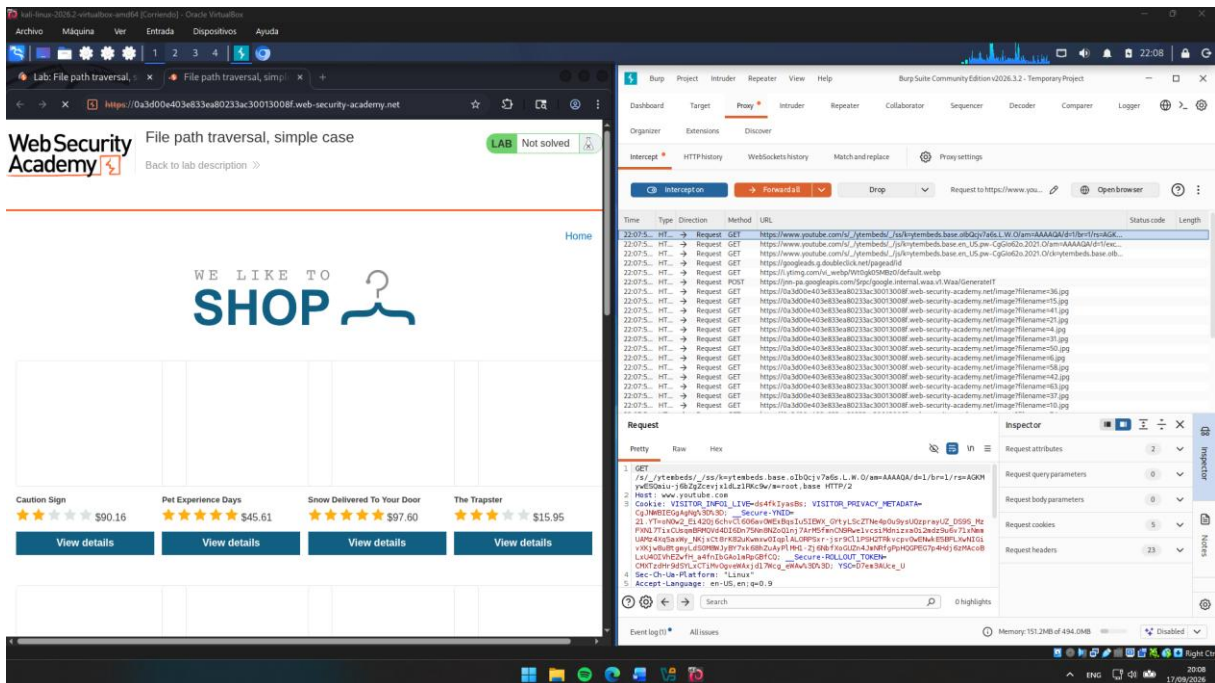
Se inició la herramienta Burp Suite y se configuró su navegador integrado para interceptar el tráfico web. El objetivo de este paso es situar a Burp Suite como un intermediario (proxy) entre el navegador y el servidor web. Esto es fundamental porque nos permite pausar, observar y analizar absolutamente todas las peticiones HTTP que ocurren en segundo plano antes de que lleguen al servidor, incluyendo aquellas que el navegador realiza automáticamente y que normalmente no vemos en la interfaz gráfica.



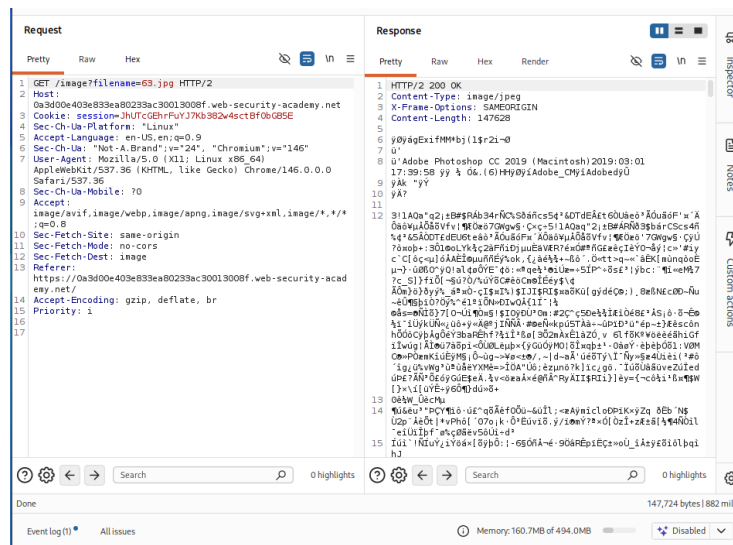
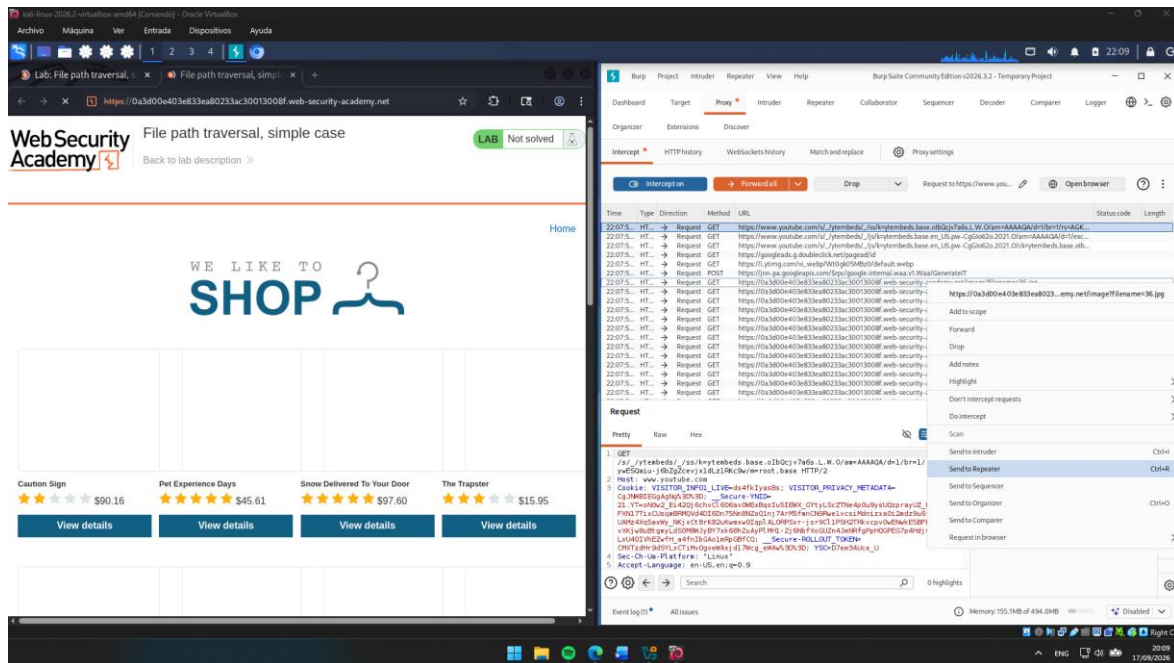
Con la opción de intercepción activada (o simplemente registrando el tráfico en la pestaña "HTTP history"), se navegó de forma habitual por la página principal de la tienda del laboratorio. El propósito de interactuar legítimamente con la aplicación es generar tráfico para que Burp Suite construya un mapa del sitio y nos revele cómo la aplicación web maneja los recursos estáticos (imágenes, scripts, estilos).



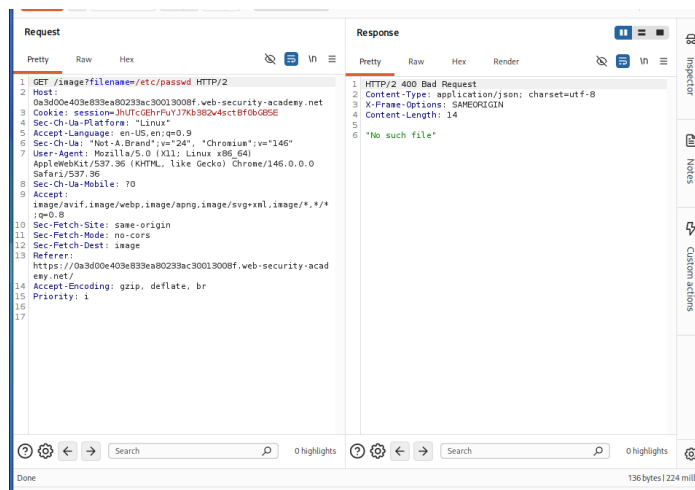
Al analizar detenidamente el historial de peticiones HTTP, se detectó una solicitud de tipo GET dirigida al endpoint `/image?filename=...` (en las capturas se observa que carga archivos como `63.jpg`). Este paso es crítico porque nos indica que la aplicación está utilizando una variable en la URL (`filename`) directamente controlada por el usuario para interactuar con el sistema de archivos del servidor. Esto lo convierte en el principal candidato para probar vulnerabilidades de acceso a archivos.



Se seleccionó esta petición específica en el historial, se hizo clic derecho y se envió a la herramienta "Repeater". La razón para usar el Repeater es que nos proporciona un entorno de pruebas aislado y controlado. Aquí podemos modificar manualmente cualquier parámetro de la cabecera HTTP de forma repetitiva sin depender del comportamiento del navegador web, lo que agiliza el proceso de ensayo y error necesario para armar el *payload* correcto.



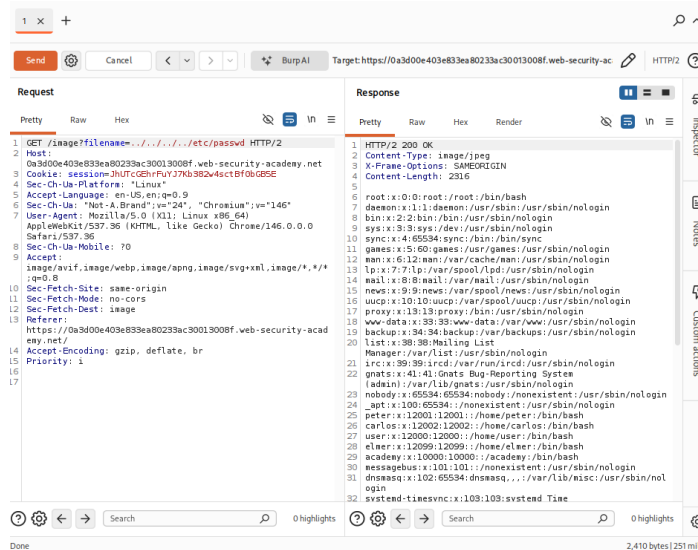
Dentro del Repeater, se modificó el valor del parámetro para solicitar una ruta absoluta directamente: filename=/etc/passwd. Al enviar esta petición, el servidor devolvió un código de estado "400 Bad Request" y el mensaje "No such file". El porqué de este resultado indica que la aplicación no está evaluando la ruta desde la raíz del sistema operativo. Probablemente está concatenando nuestra entrada a un directorio base predeterminado (ej.



/var/www/images/) y buscando un archivo que se llame literalmente "/etc/passwd" dentro de la carpeta de imágenes, el cual no existe.

Para evadir la restricción anterior y lograr "escapar" de la carpeta de imágenes predeterminada, se implementó una secuencia de salto de directorios. Se modificó el valor a filename=../../../../etc/passwd. El uso repetido de la secuencia `../` es un comando a nivel de sistema operativo que le indica al servidor que suba un nivel en la jerarquía de directorios. Al concatenar varios de estos comandos, forzamos a la aplicación a retroceder consecutivamente hasta llegar al directorio raíz (/) del servidor Linux, para luego descender hacia la carpeta /etc y localizar el archivo passwd.

Se envió la petición final modificada. En esta ocasión, el servidor respondió con un código de éxito HTTP "200 OK". En el panel de respuesta del Repeater se pudo visualizar en texto plano todo el contenido del archivo /etc/passwd (el registro de los usuarios del sistema operativo). Esto demostró que el servidor procesó los comandos de salto de directorio proporcionados en la URL, confirmando la extracción exitosa de datos sensibles y validando la resolución del



Explicación del payload

El payload utilizado fue ../../../../etc/passwd. La secuencia ../../ es directiva estándar a nivel de sistema operativo que indica subir un nivel en la jerarquía de directorios. Como se desconoce exactamente en qué subdirectorio está alojada la aplicación web dentro del servidor, se concatenan múltiples secuencias ../../ (en este caso tres) para asegurar que la ruta retroceda hasta el directorio raíz (/) del sistema de archivos. Una vez situados en la raíz, se especifica la ruta hacia /etc/passwd, un archivo estándar en sistemas Linux/Unix que almacena la información de las cuentas de usuario.

Mitigación recomendada

Para prevenir de manera efectiva las vulnerabilidades de Path Traversal, se recomienda aplicar las siguientes medidas de seguridad combinadas:

1. **Evitar la exposición directa de nombres de archivos:** La forma más segura de prevenir el ataque es no utilizar la entrada del usuario para buscar nombres de archivos físicos. En su lugar, se deben usar identificadores indirectos (como un ID numérico o un hash en una base de datos, ej. ?id=5). *Por qué:* Esto elimina el vector de ataque de raíz, ya que el usuario nunca interactúa con el sistema de archivos.
2. **Validación estricta de entradas (Lista blanca):** Si es indispensable recibir el nombre del archivo, se debe validar estrictamente contra una lista blanca. La aplicación solo debe aceptar caracteres alfanuméricos y extensiones válidas, rechazando de inmediato cualquier entrada que contenga barras (/ , \) o secuencias de salto (../). *Por qué:* Los atacantes usan codificaciones complejas para saltarse los filtros; bloquear la petición directamente es más seguro que intentar "limpiar" la cadena maliciosa.
3. **Verificación de rutas canónicas:** Antes de procesar la lectura del archivo, la aplicación debe resolver la ruta absoluta de la solicitud utilizando funciones nativas del lenguaje (como realpath()). Luego, debe verificar que esta ruta final comience estrictamente dentro del directorio base permitido (ej. /var/www/images/). *Por qué:* Esto asegura que, sin importar los saltos introducidos, la aplicación aborta la acción si el destino final intenta salir de la carpeta autorizada.
4. **Principio de menor privilegio:** Configurar los permisos del servidor web (ej. el usuario www-data en Linux) para que solo tenga acceso de lectura a los directorios estrictamente necesarios para la página. *Por qué:* Si el atacante logra evadir las validaciones a nivel de código, el propio sistema operativo bloqueará el acceso a archivos críticos como /etc/passwd por falta de permisos.